PROJECT PROPOSAL

Enforcing Booked Slots on Research-Lab PCs with a Self-Hosted Headscale Server

A 15-machine pilot

Prepared by Zamiul Rashid, Akid Anis and Tanzim Ahmed

Supervisor Md. Golam Rabiul Alam · **Co-supervisor** Mollah Md. Saif · October 2026

Summary. Every thesis group already has booked slots on our lab PCs, but nothing enforces them. Groups work remotely through AnyDesk, and anyone with a PC's AnyDesk details can connect at any time. We have built a system that enforces the slots, and tested it. A self-hosted Headscale server (an open-source control server for the Tailscale/WireGuard VPN) admits only students who sign in with their BRACU Google account. The lab's weekly routine stays in a Google Sheet. Our slot system, **labslots**, reads it every 30 seconds; at each slot change it rewrites the network policy and switches the PC to the incoming group's account, so only the group in its slot can reach its PC. Every login to a lab PC uses Tailscale SSH, so there are no keys or passwords to hand out or take back. The system already runs on our own hardware with three lab PCs. In a test with 871 simulated devices on a 4-core server, every slot change took effect within 20 seconds, and none of 2,956 login checks got in where it should not have. For a 15-PC pilot on campus we need **one public IP address, one domain name under `bracu.ac.bd`, two inbound ports (three with HTTP) and a lab email account** (Section 6).

1 Background and Goals

Each thesis group is assigned two 12-hour slots per week on a lab PC and works remotely through AnyDesk. The slots exist only on paper. AnyDesk connects through AnyDesk's own servers, so anyone who knows a PC's ID and password can connect whenever they like. That person can stop another group's training run, change its files or take over its desktop, and nobody can tell afterwards who did it. A group has one to five students, and students often use more than one device. Physical access stays under the lab's existing rules. The pilot aims to:

1. Enforce the slots: during a slot only that group can reach its PC remotely; outside it, nobody except the lab admins.
2. Run Headscale and the slot system on one machine in the lab, reachable through one public IP address and one domain name.
3. Enroll 15 Linux lab PCs, each registered once as a lab-owned machine.
4. Let students sign in with their BRACU Google account, so the lab keeps no passwords.
5. Switch access automatically at every slot change, and record every change and every enforcement step.
6. Keep the schedule where the lab already keeps it, a Google Sheet, and take group enrollments through our own page instead of a Google Form.
7. Measure reliability and load before any wider roll-out.

2 Options Considered: Why Headscale

Table 1 compares the main ways to give students remote access to lab PCs.

Table 1: Options for remote access.

Option	Control and data path	Fit for slot-based lab access
Current setup: AnyDesk only	AnyDesk's servers	No slot enforcement: anyone with a PC's ID and password can connect at any time, and there is no record of who did
SSH port-forwarding or a bastion host	Ports opened on campus; SSH exposed to the internet	Needs more than one IP and port from IT; open to brute-force attempts; slots enforced only on each PC
Plain WireGuard (hub)	Our server; all traffic through the hub	No NAT traversal; keys and firewall rules managed by hand for every device
Tailscale (hosted)	Tailscale Inc. (off campus); direct P2P	Can share a device into a group's own tailnet by email. The free plan covers 6 users; Standard costs $8 per user per month. The control plane and user data stay off campus
NetBird (self-hosted)	Ours; direct P2P	Capable, but several services to run (management, signal, relay, dashboard, identity provider) and its own clients
Headscale (chosen)	**Ours; direct P2P**	One binary plus SQLite, official Tailscale clients, free (BSD-3). The policy is one document we regenerate and load through its API at every slot change

3 System Architecture

The design separates a **control plane** from a **data plane** (Figure 1). Headscale is an open-source reimplementation of Tailscale's coordination server that works with the official Tailscale clients. It registers devices, hands out tailnet addresses and keys, and pushes each device a peer list filtered by the access policy. It never carries user traffic. The data plane is WireGuard, running directly between the student's laptop and the lab PC. Next to Headscale runs labslots: a website (the *hub*) and a *slot agent* that enforces the schedule.

How a connection is made.

1. Every device keeps an HTTPS (TCP 443) session to Headscale and receives only the peers the policy allows.
2. A new connection starts out relayed through a DERP server. Meanwhile, both ends learn their public addresses via STUN (UDP 3478) and attempt NAT traversal.
3. If a direct UDP path works, traffic moves to it (lowest latency). Under hard NAT, or where UDP is blocked, it stays on DERP over TCP 443, which is slower but works. The client keeps re-checking for a direct path.
4. Every path is WireGuard-encrypted end to end. The relay only ever sees ciphertext.

Our host runs Headscale's **embedded DERP relay**, so relayed traffic stays on campus. Tailscale's public DERP servers can stay in the map as a backup, or be removed to keep everything in-house.

One entry point. Caddy terminates TLS for `hs.cse.bracu.ac.bd` with an automatic Let's Encrypt certificate. It sends `/slots` to the labslots website and everything else to Headscale, which needs WebSocket support from the proxy; Caddy provides it, and our current deployment already runs behind a reverse proxy. STUN (UDP 3478) goes straight to Headscale. Lab PCs reach the server on its campus LAN address through a hosts-file entry, so they do not depend on the campus firewall looping traffic back to the public IP (hairpin NAT).

3.1 Machines, users and slots

Each **lab PC** is registered once with a pre-authorised key that carries its tag (`tag:pc-NN`). That makes it a *tagged node*, owned by the lab rather than by a person. Every student is a user in our tailnet, identified by their BRACU email. A **thesis group** is a list of emails in the access policy and one

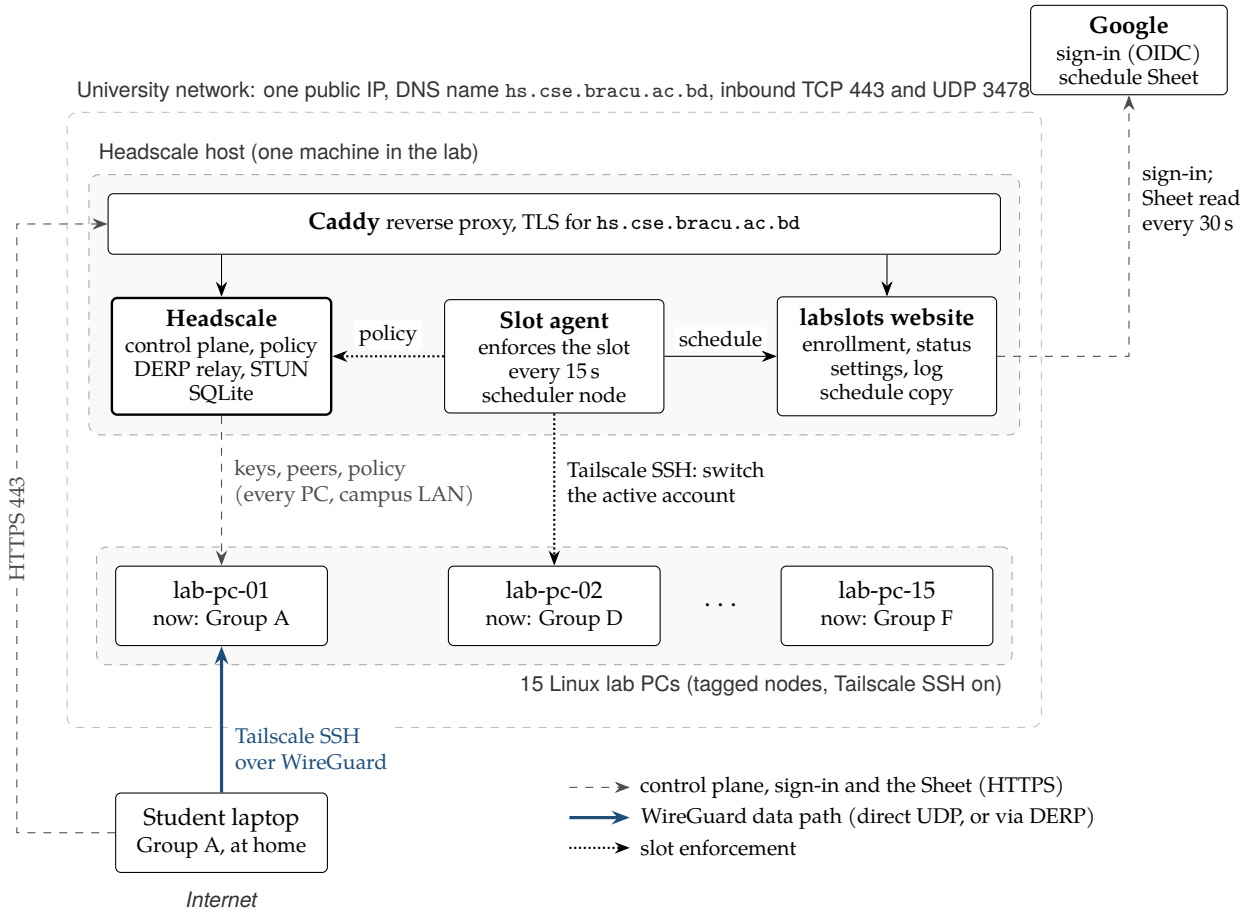


Figure 1: Pilot architecture. Caddy is the only public entry point: it serves Headscale and the labslots website under one name. User traffic flows between laptop and lab PC, end-to-end encrypted with WireGuard.

Linux account on its PC (`grp-<code>`). "Sharing" a PC with a group is a policy grant that exists only during the group's slot. The PC cannot start connections back to the students.

Each PC has 14 slots per week, **08:00–20:00** and **20:00–08:00** every day. Each group normally gets two slots on different days (Table 2). The routine is kept in a **Google Sheet**: one *Routine* tab per term with a row per PC and a column per slot, where an admin types a group's code, and tabs for groups, PCs, terms and admins. The labslots website reads the Sheet every 30 seconds and checks every cell. A bad cell keeps its last good value and is listed in the Sheet's *Problems* tab, and if Google cannot be reached the system keeps enforcing its last good copy.

The **slot agent** runs next to Headscale and acts every 15 seconds, so a missed boundary (a restart, an outage) is put right on the next run:

1. it writes the policy for the current slot and loads it through Headscale's API. All 15 PCs change in one update. Network access and Tailscale SSH logins change here, and each PC ends the outgoing group's open sessions itself;
2. it logs in to each PC as the scheduler node over Tailscale SSH and runs `slot-switch`, which makes the incoming group's account the only active one and stops the outgoing group's processes;
3. on any error, it loads an admins-only policy, so access fails closed. Having no policy at all would allow everything.

Table 2: Example weekly schedule for one PC: six groups (A–F) and one maintenance slot.

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
08:00–20:00	A	C	E	A	C	E	*maint.*
20:00–08:00	B	D	F	B	D	F	*spare*

```
"groups": { "group:thesis-a": ["s2101@g.bracu.ac.bd", "s2102@g.bracu.ac.bd", ...] },
"grants": [
  { "src": ["group:admins"], "dst": ["tag:pc-01", ...], "ip": ["*"] },
  { "src": ["tag:scheduler"], "dst": ["tag:pc-01", ...], "ip": ["tcp:22"] },
  // generated for the current slot (Mon 08:00-20:00): pc-01 shared with Group A
  { "src": ["group:thesis-a"], "dst": ["tag:pc-01"], "ip": ["tcp:22"] } ],
"ssh": [
  { "action": "accept", "src": ["group:thesis-a"], "dst": ["tag:pc-01"],
    "users": ["grp-thesis-a"] },
  { "action": "accept", "src": ["tag:scheduler"], "dst": ["tag:pc-01", ...],
    "users": ["labsched"] } ]
```

Lab PCs. Each PC runs Ubuntu LTS with the Tailscale client and **Tailscale SSH** turned on. On the tailnet, port 22 is answered by the Tailscale client itself, which checks the caller's tailnet identity against the policy's `ssh` rules: a group member logs in as the group's account, the scheduler only as `labsched`, and admins as any non-root account. Nobody holds an SSH key, and there is nothing to revoke when a student leaves. One setup script per PC turns Tailscale SSH on and installs the `slot-switch` helper. Home directories persist between slots, but running jobs stop at slot end, so groups must checkpoint.

3.2 Student sign-in and enrollment

Headscale does not check passwords itself: it hands sign-in to **Google** over OIDC, and admits only verified accounts on BRACU's domains. The labslots website signs people in the same way. The lab stores no passwords, and a student who forgets theirs uses Google's own recovery.

What the student does.

1. One member of the group signs in to the enrollment page with their BRACU Google account and sends the group's details: group name, thesis title, the supervisor and any co-supervisors with their emails, and one to five members with name, student ID and BRACU G Suite address. Personal addresses are refused. This replaces the Google Form the lab used before.
2. An admin reviews the enrollment and assigns a PC and weekly slots in one step; they are written into the Sheet. The group sees its PC, slots and how to connect on its status page, and every member is emailed once outgoing email is set up.
3. Each member runs `tailscale login --login-server https://hs.cse.bracu.ac.bd` once, or presses "Log in" in the Tailscale app, and signs in with their BRACU account.
4. During a slot: `ssh grp-a@lab-pc-01`. Outside it, the PC cannot be reached.

Signing in proves who you are; the schedule decides what you can reach. Students can keep a personal Tailscale account alongside ours, because client 1.34 and later supports several accounts.

The admin desk. Admins use the same website: the PCs in use now and what changes next, a calendar of the routine, the enrollment queue, the state of each PC, the enforcement settings with a preview of the policy, and a log of every change and enforcement step.

3.3 Devices

A *node* is a device, not a user: a student with a laptop and a phone is two nodes (Table 3).

Table 3: Nodes on the server (15 lab PCs and 1 scheduler included).

Situation	1 device per student	2 devices per student
One slot, all 15 PCs in use, 5 students per group	91	166
Devices kept all week (6 groups per PC, 90 groups)	466	916

Headscale's FAQ says it "can likely handle 100s of devices (maybe more)" when the network changes rarely, because every change rebuilds the network map for every node. Our earlier plan therefore removed every device at the end of its slot, which would make students sign in again before each slot. Our stress test (Section 4) shows this is not needed at pilot scale: a 4-core server held 871 devices that never left, and stayed almost idle between slot changes. Devices therefore stay registered; access follows the policy, not the device list. Clean-up removes only what is no longer needed:

- a day after a student leaves every group (their group or term is removed from the Sheet), their devices are removed;
- an optional switch also removes devices outside their slot and beyond two per student. It starts in dry-run mode, listing what it would remove.

3.4 Remote desktop in the slot (planned)

AnyDesk is how groups work today, and it is where the slots are bypassed. AnyDesk connects through *AnyDesk's own servers*, not our tailnet, so our policy cannot control it, and its free licence does not cover lab use. We therefore plan to turn off AnyDesk's unattended access on enrolled PCs and offer a remote desktop that runs over the tailnet instead (RustDesk in direct-IP mode, or xrdp). Its port is added to the ports a group gets in its slot, and the same policy then controls it, just like SSH. This part is not built or tested yet.

4 What We Have Built and Tested

Running today. Headscale 0.29.3 and labslots run on our own server behind a reverse proxy, at a domain of our own, with three lab PCs enrolled (pc-41, pc-43 and pc25) and Google sign-in limited to BRACU accounts. The website, the Sheet, enrollment and slot enforcement work end to end, and an automated test suite (73 tests) covers the policy, the enforcement steps, the Sheet and the website.

Home-lab rehearsal (13 October). We rebuilt the pilot on our home network, with a simulated campus, firewall and off-campus clients, measured it, and removed it again (Table 4).

Table 4: Rehearsal results against the pilot's targets.

Measure	Target	Result
Enrollment time per lab PC	$\leq$ 10 min	11 s (package already installed)
Time to revoke access	$\leq$ 1 min	3–5 s
Restore from backup on a new server	$\leq$ 2 h	63 s, all nodes back
Access-policy tests	all pass	13 of 13
Direct (not relayed) connections	$\geq$ 80%	44–58% (see Risks)
30-minute server outage	sessions survive	direct: 0% loss; relayed: 92% loss
Off campus, only TCP 443 allowed	works	relayed at 27–37 ms

Stress tests (22 October). We ran the complete server side (Headscale, website, slot agent, scheduler node) in a container and drove it with simulated lab PCs and students; run 2 spread them across three machines. Every simulated PC ran the real setup script, and every student device was a real Tailscale client. Throughout, sampled students were checked against the schedule every 20–30 seconds: their group's own PC must be reachable during the slot, other PCs must not. Run 1 checked

network access to port 22; run 2 made real Tailscale SSH logins, and also tried another group's account, which must fail. Table 5 summarises both runs.

Table 5: Stress tests. Times are from the schedule change; they include up to 15 s until the agent's next run.

	Run 1	Run 2
Server	6 cores	**4 cores**, 4 GB
Lab PCs, groups, students	70, 57, 80	70, 140, 700
Devices online at peak	181	**871** (none removed)
PC logins by the scheduler	SSH key	Tailscale SSH
Policy in force after a slot change	5–14 s	13–20 s
All 70 PCs on the new account	23–53 s	13–20 s (switching: 1.5 s)
Checks that got in where they should not have	0 of 5,037	0 of 2,956
Headscale CPU, steady	12% of one core	2% of one core
Headscale CPU, peak	1.1 cores	3.9 cores (800 devices signing in within 3 minutes)
Headscale memory, peak	161 MB	450 MB

What this means for the pilot. A 15-PC pilot needs about 100–900 devices, depending on how many students keep devices registered. A 4-core machine with 2 GB of memory or more handles that; only a mass sign-in, such as the first day of term, briefly uses all four cores. Slot changes take effect within about 20 seconds, and the policy held in every probe.

5 Technical Specifications

The whole system runs on one small, low-power machine in the lab (Table 6).

Table 6: Headscale host: hardware and power.

Machine	8-core mini PC or small desktop (AMD Ryzen 7 or Intel Core i7 class), 16 GB RAM, 512 GB NVMe SSD, 1 GbE wired. The stress test ran the server side on 4 cores and under 1 GB, but a mass sign-in used all four cores; eight leave room for that, for growth beyond 15 PCs, and for a second relay or monitoring on the same machine
Software	Ubuntu Server 24.04 LTS; Headscale 0.29.3 (pinned) with SQLite and its embedded DERP relay; Caddy; Docker running the labslots website, the slot agent, the scheduler node (Tailscale 1.102.3) and a nightly backup job
Power draw	Roughly 8–15 W idle and 60–90 W at full load, depending on the model; about 11 kWh per month at a 15 W average. We will measure our unit
Backup power and data	650 VA UPS. Nightly copies of the Headscale, website and agent databases (kept 14 days), with Headscale's keys and configuration, go to a second disk

Section 6 lists the ports exposed to the internet. On the lab PCs, SSH is answered only on the tailnet, by Tailscale SSH.

6 What We Need from the University

We ask for the five things in Table 7. The project team provides everything else: the machine, software, certificates, setting up the lab PCs, and running the system.

Table 7: What we need from the University.

	Item	Exactly what	What it is for
1	Public IP address	One static public IPv4 address, routed or forwarded to the Headscale host in the lab	Students reach the server from home and campus
2	Domain name	One DNS record under `bracu.ac.bd` for that address, for example `hs.cse.bracu.ac.bd` (the exact name is IT's choice)	Headscale and the labslots website, with a Let's Encrypt certificate
3	Firewall ports	Inbound to that address only: TCP 443 and UDP 3478, and TCP 80 if allowed (Table 8)	Control traffic, the relay, STUN and certificate checks
4	Lab email account	One account on `bracu.ac.bd` for the lab, for example `cse-labslots@bracu.ac.bd`, with an app password or permission to send through BRACU's Google Workspace SMTP relay	Emails each group the result of its enrollment
5	Google sign-in	If BRACU's Google Workspace restricts third-party apps: our Google OAuth client allowed for BRACU accounts	Students and admins sign in with their BRACU accounts

Table 8: Ports exposed to the internet (inbound to the public IP only).

Port	Received by	Purpose	Status
TCP 443	Caddy	Tailscale control traffic and the DERP relay (Headscale), and the labslots website	Required
UDP 3478	Headscale	STUN, so devices can find their public address and connect directly	Required
TCP 80	Caddy	Let's Encrypt certificate checks; redirects browsers to HTTPS	Recommended*

*Caddy can also validate certificates over TCP 443, so port 80 can stay closed if the University prefers.

Nothing else is exposed. The lab PCs, port 22 and the services behind Caddy are not reachable from the internet. Student traffic reaches a PC only through WireGuard, and only during the group's slot.

Outbound traffic from the lab network, which campus networks normally already allow: UDP 41641, UDP 3478 and TCP 443 to any address, from the lab PCs and the Headscale host. If outbound UDP is blocked, connections still work through our relay on TCP 443, only slower.

7 Risks

Risk	Mitigation
The slot agent cannot build or load the policy	Falls back to an admins-only policy (no student access); the error shows on the admin desk and in the log
The website or the Google Sheet is unreachable	The slot agent keeps enforcing its last good copy of the schedule; edits wait until the Sheet can be read again
A mistaken edit in the Sheet	A bad cell keeps its last good value and is listed in the Problems tab; removing more than half of a tab at once is held back; devices are removed only a day after a student leaves every group
Headscale restarts or goes down	Direct connections survive (0% loss in a 30-minute outage) but relayed ones drop. Restarts and upgrades are done between slots; a relay on a second machine removes this if needed
Fewer direct connections than hoped (44–58% in the rehearsal)	Relayed connections still work. Lab PCs behind one NAT all use UDP 41641; giving each PC its own port, or an endpoint-independent NAT, improves this
Many devices sign in at once (first day of term)	Measured: 800 devices in about 3 minutes on 4 cores, all online; slot changes are not affected afterwards
BRACU's Google Workspace blocks our sign-in	We ask IT to allow our OAuth client; the admins keep access through their own accounts meanwhile
A student shares their account	The account still reaches only that group's PC during its slot; an admin can remove the device at any time
Remote desktop outside the tailnet (AnyDesk)	Unattended AnyDesk access is turned off on enrolled PCs; remote desktop runs over the tailnet under the same policy
Headscale host fails	UPS and nightly backups; a restore on a new machine took 63 seconds in the rehearsal